

LITHE: A Query Rewrite Advisor using LLMs

Sriram Dharwada
sriramd@iisc.ac.in
Indian Institute of Science
India

Jayant R Haritsa
haritsa@iisc.ac.in
Indian Institute of Science
India

Himanshu Devrani
himanshud@iisc.ac.in
Indian Institute of Science
India

Harish Doraiswamy
harish.doraiswamy@microsoft.com
Microsoft Research India
India

Abstract

When complex SQL queries suffer slow executions despite query optimization, DBAs typically invoke automated query rewriting tools to recommend “lean” equivalents that are conducive to faster execution. The rewritings are usually achieved via transformation rules, but these rules are limited in scope and difficult to update in production systems.

We investigate here how the remarkable cognitive capabilities of LLMs can be leveraged for performant query rewriting while incorporating safeguards and optimizations to ensure correctness and efficiency. Our study shows that these goals can be progressively achieved through incorporation of (a) an ensemble suite of basic prompts, (b) database-sensitive prompts via redundancy removal and selectivity-based rewriting rules, and (c) LLM token probability-guided rewrite paths. Further, a suite of logic-based and statistical tools can be used to check for semantic violations in the rewrites prior to DBA consideration.

We have implemented the above LLM-infused techniques in the LITHE system, and evaluated complex analytic queries from standard benchmarks on contemporary database platforms. The results show significant performance improvements for slow queries, over both SOTA rewriters and the native optimizer. For instance, with TPC-DS on PostgreSQL, the GM of runtime speedups was a high 13.2 over the native optimizer, whereas SOTA only gave 4.9.

Overall, LITHE is a promising step toward viable LLM-based advisory tools for enhancing enterprise query performance.

Keywords

SQL Query Rewrite, LLM, Prompt Engineering, MCTS

1 Introduction

The SQL queries embedded in enterprise applications are often riddled with inefficiencies and redundancies [2, 6, 47], especially when machine-generated by modeling software such as ORM tools (e.g., Entity Framework [36], Hibernate [27]). Query optimizers are expected to, in principle, automatically remove such wasteful bloat while constructing efficient execution plans. However, in practice, they are often led astray by complex representations, resulting in poor response times.

As a case in point, consider the blog-processing query [6] shown in Figure 1, which computes a daily summary of rating metrics for highly-rated blogs. When this seemingly complex query is given to the current PostgreSQL engine (v16) [3], its execution plan essentially mimics the hierarchical structure of

```
SELECT t.Key, sum(t.Rating) AS PostRating,
       (SELECT sum(b0.Rating)
        FROM (SELECT p0.PostId, p0.BlogId, p0.Content,
                    p0.CreatedDate, p0.Rating, p0.Title,
                    b1.BlogId AS BlogId0, b1.Rating AS Rating0,
                    b1.Url, p0.day AS Key
               FROM Posts AS p0 INNER JOIN Blogs AS b1
                ON p0.BlogId = b1.BlogId
               WHERE b1.Rating > 5) AS t0
        INNER JOIN Blogs AS b0 ON t0.BlogId = b0.BlogId
        WHERE t.Key = t0.Key ) AS BlogRating
FROM (SELECT p.Rating, p.day AS Key
      FROM Posts AS p INNER JOIN Blogs AS b
       ON p.BlogId = b.BlogId
      WHERE b.Rating > 5) AS t
GROUP BY t.Key;
```

Figure 1: Complex SQL Representation

```
SELECT p.day AS Key, SUM(p.Rating) AS PostRating,
       SUM(b.Rating) AS BlogRating
FROM Posts AS p INNER JOIN Blogs AS b
 ON p.BlogId = b.BlogId
WHERE b.Rating > 5
GROUP BY p.day;
```

Figure 2: Lean Equivalent Query

the query. This strategy leads to multiple scans and joins of the Blogs and Posts tables, making the query take several *minutes* to complete. However, the query can be equivalently rewritten (assuming NOT NULL column constraints and key-joins) in the “lean” flat version shown in Figure 2 – this alternative requires only a single join, and runs more than an *order-of-magnitude* faster, completing within seconds!

For such optimizer-failure scenarios, an alternative option to rectify slow-running queries is to carry out *external tuning* – in particular, DBAs typically resort to SQL-to-SQL (S2S) *query rewriting tools* for recommending performant alternatives. A viable S2S query transformer should cover a wide range of queries and ensure the rewrite is: (1) semantically equivalent to the original; (2) ideally performance-beneficial, but at least not causing regression; and (3) incurring acceptable overheads.

Prior Work

A range of innovative *Rule-based* (e.g. Learned Rewrite [53]) and *Model-based* (e.g. Gen-Rewrite [33]) approaches have been proposed for S2S transformations. These state-of-the-art (SOTA) techniques have foregrounded the potential benefits of query rewriting. However, as explained later in Section 9, their realization of these benefits is curtailed by: (a) restrictions in rewrite

scope, (b) susceptibility to semantic and syntactic errors, and (c) transformations via the plan space (i.e. optimization on the nodes of the execution plan tree) rather than directly in query space (i.e. transforming the query structure itself). In this paper, we address these lacuna and substantively amplify the effectiveness of query rewriting.

The LITHE Rewriter

We propose LITHE (LLM Infused Transformations of HEfty queries), an LLM-based query rewriting assistant to aid DBAs in tuning slow-running queries. As illustrated in the architectural diagram of Figure 3, LITHE takes the user query Q^U as input and outputs a transformed query Q^T , together with (a) the *expected* performance improvement of Q^T , in terms of optimizer estimated cost; (b) a *verification label* indicating the semantic equivalence mechanism (provable or statistical) between Q^T and Q^U ; and (c) a *reasoning* for why the LLM expects Q^T to be helpful wrt performance. Armed with this information, DBAs can leverage their expertise to decide whether or not to use Q^T . Note that having the DBA in the loop is a common practice in commercial query advisory systems [17].

To help the LLM adapt to different query patterns and structures, we introduce database-aware rules into the prompts, which make use of the rich metadata (e.g., logical schema, selectivities) available in database environments (Section 4). Our rules are invoked directly in *query space*, providing the LLM with the latitude to generalize the rule usage to a wide range of queries. As shown later in our experiments, this is a powerful mechanism for ensuring performant rewrites across database environments. Furthermore, we leverage the rich telemetry provided by LLMs – particularly, the *token probabilities* output at each step in the prediction sequence. Whenever the LLM lacks high confidence in the next token, we follow multiple alternative paths in the decision process. To ensure practical overheads on this enumerative approach, a Monte Carlo tree search (MCTS) technique is incorporated in our implementation (Section 5).

LITHE also incorporates a suite of logic-based and result equivalence-based semantic equivalence tests to verify the correctness of the rewrites (Section 6.2). Finally, given that there are often material discrepancies between optimizer cost predictions and actual execution times [34], LITHE includes heuristic mechanisms, described in Section 6.3, to identify “brittle” rewrites and discard them in favor of the original.

Results. Our experiments on benchmark environments demonstrate that LITHE achieves, for many slow queries, semantically correct transformations that significantly reduce the abstract (optimizer-estimated) costs. In particular, for TPC-DS, LITHE constructed “highly productive” ($> 1.5\times$ estimated speedup) rewrites for as many as 26 queries, whereas SOTA promised such rewrites for only about half the number. Further, the GM (Geometric Mean) of LITHE’s cost reductions reached **11.5**, almost double the **6.1** offered by SOTA.

We also evaluated whether the projected cost reductions translated into real execution speedups. Here, we find that LITHE is indeed often substantively faster at run-time as well. Specifically, the geometric mean of the runtime speedups for slow queries was as high as **13.2** over the native optimizer, whereas SOTA delivered **4.9** in comparison. Finally, our regression identification techniques were successful in guarding against “brittle” rewrites that are promising cost-wise, but eventually poor at run-time.

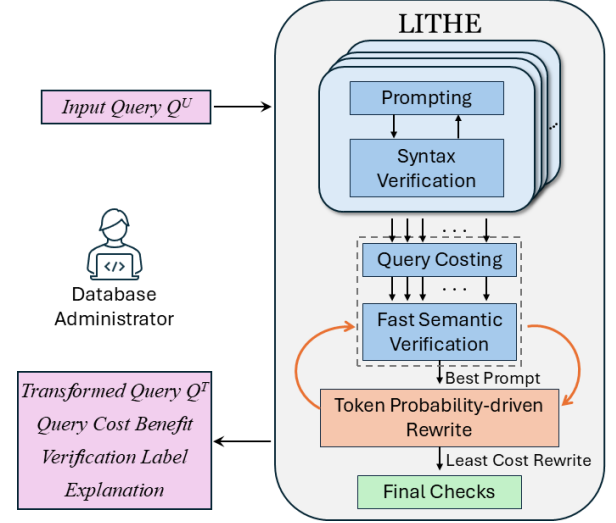


Figure 3: High-level architecture of LITHE

Contributions

In summary, our study makes the following contributions:

- (1) Assesses LLM suitability for S2S transformation.
- (2) Transforms directly in query space instead of plan space intermediates, leading to performant rewrites.
- (3) Incorporates database-sensitive rules in LLM prompts, covering both schematic and statistical dimensions.
- (4) Leverages LLM token probabilities to guide navigation of the rewrite search space and minimize LLM errors.
- (5) Evaluates rewrite quality over a range of database environments, demonstrating substantial performance benefits.
- (6) Identifies learnings that could help guide research directions for industrial-strength query rewriting.

2 LITHE Overview

The LITHE architecture (Figure 3) comprises a five-stage pipeline: **1. Prompt-based rewriting.** This component consists of two modules: LLM Prompting and Syntax Verification. The user query and an initial prompt are fed to the LLM prompting module, requesting a rewrite. The LLM output’s syntax is then verified with the database engine’s parser – if invalid, the error is returned to the LLM via an updated prompt asking for a correction. This goes on iteratively (within a threshold) until a valid SQL query is obtained.

This process is repeated for each of the different prompts described in Sections 3 and 4, and the resulting syntactically-valid rewrites are fed to the query costing component. We evaluate each prompt within a fresh LLM context, thereby making the prompt processing to be order-independent.

2. Query Costing. The costs of candidate rewrites are evaluated via the database engine’s optimizer. Rewrites whose costs are greater than that of the original query are immediately discarded. Whereas, the potentially beneficial rewrites (if any) are checked for semantic equivalence to the original query.

3. Fast Semantic Equivalence. Statistical (result-based) techniques are employed to quickly and cheaply assess the semantic equivalence of a recommended rewrite. If the rewrite is deemed valid by this module, it is returned along with the prompt that generated it; otherwise an invalid label is returned.

4. Token probability-driven rewrite. The prompt producing the most beneficial (and valid) rewrite is used as input to a Monte-Carlo tree search (MCTS)-based procedure to further refine the rewrite quality. Specifically, multiple exploration paths are followed whenever the LLM lacks high confidence in the predicted token (details in Section 5). The query costing and semantic equivalence modules are also used internally within this procedure.

5. Final Checks and Output. The least expensive valid rewrite (as identified by the MCTS module) is tested using a suite of provable (logic-based) techniques to generate a final equivalence label (provable or statistical), and its cost benefit over the original query is computed. Further, the execution “brittleness” is assessed using the robustness heuristics. If no valid rewrite is identified, or if the rewrite is expected to be a regression, the original query itself is returned to the DBA. Whereas, if a beneficial rewrite is recommended, an LLM-generated reasoning for the expected performance improvement is also extracted.

2.1 Performance Framework

We consider a query that takes more than T seconds to complete on the native database engine as a “slow query”, potentially triggering intervention by the DBA. Based on common industry perception (e.g. [7]), T is set at 10 seconds in our study. For this context, we define a **Cost Productive Rewrite (CPR)** as a rewrite that improves a slow query’s performance by at least 1.5 times wrt the optimizer-estimated cost – this aggressive choice of threshold is so that: (a) there is enough headroom in the optimizer prediction that a runtime regression is unlikely; and (b) the benefits of the rewrite substantively outweigh the transformation overheads.

The overall benefit of a rewriting tool is quantified by the number of CPR obtained on the slow query workload. Additionally, we measure **CSGM**, the *geometric mean* (GM) of the cost speedups obtained by these rewrites. Finally, to assess the actual run-time benefits, we evaluate **TSGM**, the geometric mean (GM) of the response-time speedups obtained by these rewrites. These speedups are measured relative to the original query on the native optimizer.

2.2 Query Micro-benchmark

To motivate the progression of strategies incorporated in LITHE, we create an initial micro-benchmark comprising 10 diverse TPC-DS queries for which we were able to *hand-craft* high-quality CPRs. These queries are processed on GPT-4o, the popular OpenAI LLM, and the rewrites are evaluated on the PostgreSQL v16 database engine. The human rewrites deliver a CSGM of **11.84**, serving as an aspirational target to attain computationally. Later, in Section 7, we extend the evaluation to complete benchmarks.

Further, for ease of presentation, we focus on the CSGM metric in the LITHE design sections (Sections 3, 4, 5). The TSGM performance is subsequently discussed in Section 7.

3 Basic Prompts

In this section, we explore the simplest interface to LLMs, namely *prompting*, for query rewriting. We evaluate four basic prompts, listed in Figure 4, which cover a progressive range of instructional detail and test the effectiveness of the LLM’s base knowledge.

Prompt 1: This is the baseline prompt used in [33], which simply asks the LLM to rewrite a given query to improve performance.

Prompt 2: Explicit instructions are included to maintain semantic and functional equivalence while rewriting.

Prompt 1	**Enter Query here** Rewrite this query to improve performance.
Prompt 2	You are a database expert and SQL optimizer. Your role involves identifying inefficient SQL queries and transforming them into optimized, functionally equivalent <Database Engine> versions. **Enter Query here** Rewrite this query to improve performance of query while maintaining semantic equivalence.
Prompt 3 / Prompt 4	You are a database expert and SQL optimizer. Your role involves identifying inefficient SQL queries and transforming them into optimized, functionally equivalent <Database Engine> versions. Your tone is analytical and instructional. A user has provided the following <Database Engine> query that is potentially inefficient: **Enter Query here** The task is to first identify whether the query is inefficient or not. If it is inefficient, you must rewrite the query to make it more efficient while maintaining semantic equivalence. Here are a few steps that can help you complete the task: 1. Start by identifying specific inefficiencies in the provided query. If you feel the original query is already efficient, skip the next two steps, and simply return the query as-is. 2. Next, provide guidelines for optimizations, and explain the rationale behind the recommended optimizations. Correspond how these changes would map onto the original query to maintain syntactic and semantic equivalence. 3. Finally craft the new, optimized <Database Engine> query which includes all the enhancements discussed. Give complete rewritten query with no manual involvement by user.

Figure 4: Templates used for Basic Prompts

Prompt 3: Verbose instructions are given to rewrite the query, providing step-by-step guidance to the LLM to think rationally. It is first asked to pick out potential inefficiencies in the input query, and then tasked to identify approaches to address these inefficiencies. Finally, it is instructed to apply the identified solution. Essentially, the prompt tries to make the LLM reason akin to human experts.

Prompt 4: The sequence of instructions in Prompt 3 is split into sub-prompts, and provided to the LLM in an *iterative* manner instead of all at once. This breaks down the complex instructions of Prompt 3 into digestible steps that help the LLM focus on individual tasks.

Performance. Table 1 presents the performance of the four prompt templates on the micro-benchmark. We find that less than half the rewrites are productive with individual prompts. However, a drill-down shows that the best prompt in the ensemble is *query-specific*, opening up the possibility of using all four prompts in parallel, and then choosing the best among them. This ensemble approach raises the CPRs to **6** (Row 5 in Table 1) – however, there remain four queries not productively rewritten by any prompt.

The CSGM is at most 3 for the individual prompts, whereas the ensemble reaches **3.23**. But these speedups, although productive, are all much lower than those delivered by the human rewrites.

Finally, an ensemble of SOTA techniques (details in Section 7) was processed on the same platform. They delivered 3 CPRs with a CSGM of 2.49 (last row in Table 1), highlighting the wide gap between the current reality and what is humanly possible.

Table 1: Performance of Basic Prompts.

Prompt	# CPR	CSGM
Prompt 1	3	1.99
Prompt 2	2	1.85
Prompt 3	4	2.83
Prompt 4	4	3.00
Prompt Ensemble	6	3.23
SOTA Ensemble	3	2.49

4 Database-Sensitive Prompts

As discussed above, basic prompting needs improvement on two fronts: (1) Ensuring productive rewrites where feasible; and (2) Maximizing the impact of these productive rewrites. Our discussions with industry experts revealed that query inefficiencies are commonly attributed to redundant operations within the queries themselves. Contemporary (non-LLM) optimizers do try to reduce such redundancies during the initial logical optimization phase, prior to physical plan enumeration (e.g. redundant join elimination [9]). But our analysis of execution plans for slow queries revealed that they often fail to comprehensively do so.

To address these issues, we incorporate database domain knowledge into the prompts. Specifically, we design a *one shot*-based prompting template, augmented with a set of database-aware rewrite rules. As a proof of concept, we explore two categories of rewrites here: (a) Rules that eliminate redundancy in the input queries; and (b) Predicate selectivity-based rules that implicitly guide, via query space reformulations, the query optimizer towards efficient query execution plans. Of course, this basic set of rules can be expanded further, but as shown by our experiments, even this minimal set is capable of delivering substantive improvements over a broad set of database environments.

The templates used for these prompts (Figure 5) apply only a *single* rule. This was a conscious design choice because LLMs can be overwhelmed by excessive information given in monolithic form [43]. A related question is whether, while retaining the one-rule-per-prompt design, the rules could be *progressively* applied, thereby benefiting queries with multiple types of redundancies. However, the combinatorially large number of rule permutation sequences makes the selection process appear infeasibly expensive. Therefore, we apply each rule using a separate prompt, finally returning the rewrite providing the best improvement. Despite this restrictive policy, highly performant rewrites are obtained in our evaluation (Section 7).

4.1 Redundancy Removal

There are different types of redundancy that can occur in a SQL query – repeated computations, superfluous filter predicates, unnecessary joins, etc. Rules **R1 through R4** in Table 2 are designed to tackle such redundancies. The relevant schematic information (e.g. table names, column names, constraints) required by these rules is also provided in the prompt.

The template for such rule-based prompts is shown in Figure 5(a) and includes an *example rewrite* to demonstrate the rule application to the LLM. Our empirical observation is that the precise wording of the rule instructions is not significant. Instead, what matters are the example rewrites, which ensure the LLM applies the rules effectively, and does not go off on tangential trajectories. The specific examples used with our rules are listed in the technical report [18].

Table 2: Rules for Database-sensitive prompts

	Redundancy Removal Rules
R1	Use CTEs (Common Table Expressions) to avoid repeated computation of a given expression.
R2	When multiple subqueries use the same base table, rewrite to scan the base table only once.
R3	Remove redundant conjunctive filter predicates.
R4	Remove redundant key (PK-FK) joins.
	Statistics-based Rules
R5	Choose EXIST or IN from subquery selectivity (high/low).
R6	Pre-filter tables involved in self-joins and with low selectivities on their filter and/or join predicates. Remove any redundant filters from the main query. Do not create explicit join statements.

Performance. The performance improvement achieved on the micro-benchmark by an ensemble that adds the redundancy-removing prompts to the basic set (Section 3) is shown in Table 3. We observe that the CPR increases to 7, and CSGM grows to **6.85**.

Table 3: Performance with Redundancy Removal Rules

Prompt	# CPR	CSGM
Basic Prompts $\cup \{R1, \dots, R4\}$	7	6.85

(a)	Redundancy-Removal Prompt	You are a database expert and SQL optimizer. Your role involves identifying inefficient SQL queries and transforming them into optimized, functionally equivalent <Database Engine> versions. Your tone is analytical and instructional. This is a task to rewrite queries to improve performance using the following rewrite rule: **Enter Rewrite Rule here** [ORIGINAL] **Enter Example Original Query here** [REWRITTEN] **Enter Example Rewritten Query here** Now consider the query below and try to rewrite it to improve the performance. [ORIGINAL] **Enter User Input Query here** [REWRITTEN]
(b)	Statistics Guidance Prompt	You are a database optimizer and your task is to analyze the given <Database Engine> query using database schema and statistical information available to you. Find inefficiency in the query. If query is already efficient then return the original query. Otherwise, rewrite the query in a more efficient way to improve its performance, and explain your rewrites. This is a task to improve query performance using the following rewrite rule: **Enter Rewrite Rule here** Here is an example to help you [ORIGINAL] **Enter Example Original Query here** [STATISTICS] **Enter Statistics for example Query here** [REWRITTEN] **Enter Example Rewritten Query here** Now consider the query and statistics given below and try to rewrite the query to improve performance. [ORIGINAL] **Enter User Input Query here** [Statistics] **Enter Input Query statistics here** [REWRITTEN]

Figure 5: Templates for Database-sensitive Prompts

4.2 Selectivity-based Guidance

We now consider rules whose applicability to a query is conditional on the specific database environment, specifically its *statistical* aspects. For example, consider the alternative rewrites shown in Figure 6 using the EXIST and IN clauses (highlighted in red), respectively. Here the appropriate choice is dictated by the *selectivity* of the inner subquery – EXISTS for high selectivity values and IN for low values. This observation is encoded in Rule R5 of Table 2. Similarly rule R6 pre-filters tables involved in self-joins and with low selectivity filter and/or join predicates. Note that a specific instruction to not create explicit joins is added in rule R6. This is because in the presence of CTEs, the LLM is prone to schematic confusion regarding which attribute belongs to which table, leading it to construct *invalid joins*.

The input prompt for these rules, as shown in Figure 5(b), is modified to include the following:

- (1) Estimated selectivities of columns appearing in the WHERE and JOIN clauses – these values are obtained via calls to the cardinality estimation modules of the query optimizer.
- (2) Clause rewrite rules and instructions based on statistics.
- (3) Examples relevant to the chosen rewrite rules.

Performance. The performance improvements following addition of selectivity-guided prompts are shown in Table 4. We observe that CPRs are now obtained for 9 of the ten micro-benchmark queries. Moreover, the resulting CSGM increases to 10.57, quite close to the human target of 11.84.

Table 4: Performance of Metadata-infused Prompts

Prompt	# CPR	CSGM
Basic Prompts $\cup \{R1, \dots, R6\}$	9	10.57

We note in closing that rules R1 through R6 not only add queries to the productive category, but also deliver greater improvement for those already deemed to be productive via the standard prompts of Section 3. Further, to minimize selection overheads, a classifier could be designed to choose the appropriate rule – this option is examined in the technical report [18].

5 Token Probability Driven Rewrite

A key challenge with LLMs is “hallucinations” – responses that range from being mildly incorrect to completely made up. This is often due to the output tokens having low confidence, which “confuse” the LLM and generate suboptimal outputs [24]. To have a robust approach for such cases, we take inspiration from the code generation literature [51]. Specifically, we propose a *Monte Carlo Tree Search* (MCTS) based decoding approach to search for a sequence of LLM-generated tokens that results in both a valid query rewrite and performance improvements. While MCTS has been previously used for *ordering* rule applications in query rewrites [53], our goal is to guide the LLM in *exploring* these rewrites.

This approach models the problem of query rewriting as a decision tree denoting a *Markov Decision Process* (MDP) [38]. The root node of the tree corresponds to the initial prompt. Figure 7 illustrates one such example tree. An edge from a parent node to a child represents a possible token generated by the LLM and is associated with a value denoting the probability of generating this token given the path taken thus far. This is illustrated by the text (token) and number (probability), on the left and right

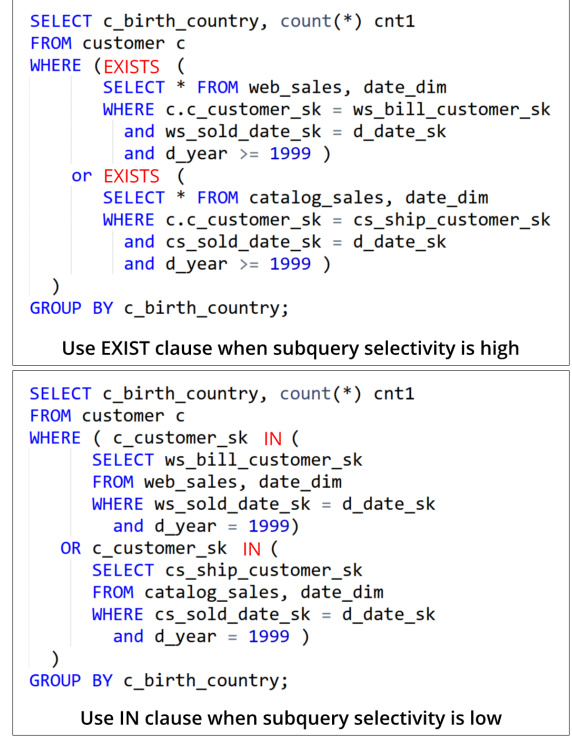


Figure 6: Example Queries illustrating Rule 5

side respectively, of an edge in Figure 7. Here, each edge can be considered as an *action* of the MDP. A node is considered *terminal* if the incoming edge corresponds to the “,” token, signaling end of the textual query.

The *state* of a node n is represented by the partial rewrite created by following the path from the root to n – it is obtained by concatenating the tokens on this path. The root’s state is an empty rewrite, and each terminal node’s state is a complete rewritten query. For example, the state of Node 4 in the decision tree shown in Figure 7 is the partial rewrite “select * from”.

Given the hundreds of thousands of tokens in LLM vocabularies, it may be very expensive (both financially and computationally) to construct the entire tree. It is therefore essential to reduce the token search space while exploring the tree for valid rewrites. This is precisely the purpose of MCTS which applies an *Upper Confidence Bound* (UCB) heuristic [41] to identify the best paths in a tree without computing the entire tree.

5.1 MCTS Search Process

The search procedure is codified in Algorithm 1, and comprises four stages – *Selection*, *Expansion*, *Simulation* and *Back Propagation* – described below, which are repeated for $iter_{max}$ iterations.

1. Selection: The first stage identifies the most suitable node of the decision tree that is yet to be expanded (i.e., the tokens corresponding to this node have not yet been processed by the search procedure). Starting from the root, this process greedily selects successive edges (actions) till an unprocessed non-terminal node is reached (Lines 6–8 in Algorithm 1). Actions are picked using a UCB that balances exploration and exploitation. The goal is to pick those actions that have either (1) a higher potential to produce correct and faster rewrites (exploitation); or (2) been selected fewer times in the past (exploration).

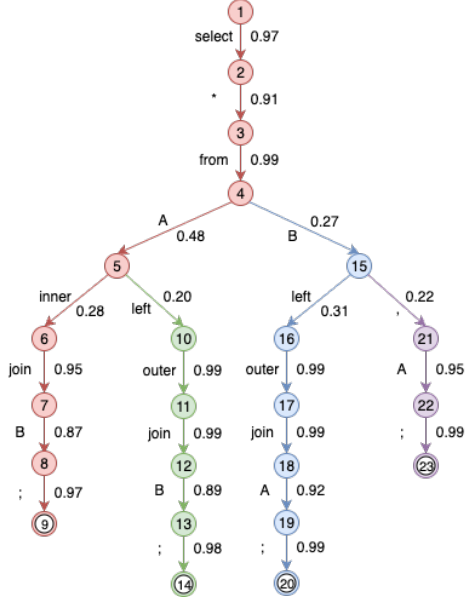


Figure 7: Sample MCTS decision tree. Paths with the same color correspond to path taken when the next token with the highest probability is used in a greedy fashion.

Specifically, given a node n and a set of possible actions $a \in A$, the next node in this traversal is chosen as:

$$n_{\text{next}} = \arg \max_{a \in A} UCB(n, a) \quad (1)$$

where UCB is a heuristic sourced from [41], modified to reflect our formulation where values are associated with nodes rather than edges in the tree. It is defined as follows:

$$UCB(n, a) = V(n') + \beta(n) * P_{LLM}(a|n.state) * \frac{\sqrt{\log(visits[n])}}{1 + visits[n']} \quad (2)$$

Here, n' is the node reached from n by taking action a , and the first component $V(n')$ represents the exploitation potential of n' to produce correct and faster queries (this notion is formalized below in Stage 3). The second component in the equation represents exploration – it is higher for those child nodes of n that are visited less often. Here, P_{LLM} represents the next token probability, which can be queried from the LLM by enabling the logprobs option in the API, and $visits[n]$ is the number of times n has been visited during the search process. β is a function that controls the balance between exploration and exploitation. It depends on two hyperparameters c_{base} and c – a higher value of c_{base} makes the algorithm favor exploitation, whereas a higher value of c increases the incentive to explore. β is defined as:

$$\beta(n) = \log\left(\frac{visits[n] + c_{base} + 1}{c_{base}}\right) + c \quad (3)$$

For example, consider the state of the tree in Figure 7 when Nodes 5 and 15 are the current unexpanded nodes. At this juncture, the selection procedure will use the UCB values of these two nodes to choose which node to expand next.

2. Expansion: The second stage is used to expand the unprocessed node n_{cur} chosen by the Selection stage. Specifically, it retrieves from the LLM the top k probable next tokens from n_{cur} 's state (Line 12), and expands the decision tree by adding k new child nodes corresponding to these tokens. To make the

Algorithm 1 Token-augmented Rewrite

```

root      # Start State
k         # Maximum number of child node expansions
θ         # Probability threshold for node expansion
itermax  # Maximum number of iterations

1: Potential, visits, V ← empty Map
2: for  $i \leftarrow 1, 2, \dots, iter_{max}$  do
3:   visits[root] ← visits[root] + 1
4:   ncur ← root
5:   # Stage 1: Selection
6:   while len(ncur.children) > 0 do
7:     ncur ← arg maxa ∈ Actions(ncur.children) UCB(ncur, a)
8:     visits[ncur] ← visits[ncur] + 1
9:   # Stage 2: Expansion
10:  expand ← True
11:  while expand and ';' ∉ ncur.state do
12:    tokensnext, Pnext ← Model(ncur, k)
13:    if Pnext[0] ≤ θ then
14:      for token ∈ tokensnext do
15:        nnew ← new Node with State ncur.state ·
          token
16:        Append nnew to ncur.children
17:        expand ← False
18:    else
19:      ncur.state ← ncur.state · tokensnext[0]
20:  # Stage 3: Simulation - Expand from ncur to full rewrite
21:  query ← GreedyExpand(ncur)
22:  v ← ComputePotential(query)
23:  Potential[query] ← v
24:  # Stage 4: Backpropagation
25:  while ncur ≠ Null do
26:    V[ncur] ← max(V[ncur], v)
27:    ncur ← Parent(ncur)
28:  # Return valid rewrite with maximum Potential > 1
29:  if ∃ q ∈ Potential | Potential[q] > 1 then
30:    return q having maximum value of Potential[q]
31:  else
32:    return the original query
  
```

expansion tractable, multiple child nodes are added only if the probability of the highest token falls below a threshold θ (Line 13). Otherwise, tokens are generated in a greedy fashion from the current node until when the LLM is again unsure of the next token, or it reaches a terminal node (i.e. completes a query rewrite).

In our implementation, k and θ are set to 2 and 0.7, respectively. Since branching occurs *only* when this relatively high θ threshold is breached, most nodes have only *one* child, resulting in long linear sequences. This design, combined with at most two nodes being processed during a branch, significantly reduces the effective fan-out of the tree. Moreover, it ensures that MCTS focuses computational resources on the brittle segments of the rewrite, and does not waste them on robust modifications. For example, when the root node in Figure 7 is first processed, then Nodes 1, 2, 3 and 4 are expanded and created sequentially since the token probabilities are higher than θ (Lines 24–26 of the while loop). Only when Node 4 is reached, two new nodes (5 and 15) are created as its children.

3. Simulation: The node n_{cur} is expanded greedily based on the highest-probability tokens until a terminal node is reached

(Line 21). The rewritten query obtained from this greedy expansion is then used to compute the potential $V(n_{cur})$ (Line 22) as follows: for a valid rewrite, $V(n_{cur})$ represents the *speedup* it provides w.r.t. the original query. However, if invalid (i.e. syntactically or semantically incorrect), $V(n_{cur})$ is assigned a zero value. After every simulation, the rewritten query obtained after the greedy expansion of n_{cur} is cached along with $V(n_{cur})$ in a map, *Potential* (Line 23).

For instance, to compute $V(n_{15})$ in Figure 7, the blue path in the tree is greedily expanded to identify its rewrite potential. That is, the partial rewrite “select * from B left outer join A;” is used to evaluate $V(n_{15})$.

4. Back Propagation: The V value of the simulation for n_{cur} is back-propagated to all its ancestor nodes, and their values are updated iff the new value is higher than the existing value.

Rewritten Query. At the end of all iterations, $q \in \text{Potential}$ with highest value $\text{Potential}[q]$ that is greater than 1 is returned as the rewritten query (Lines 29–30). In case no such rewrite exists, implying that all the valid rewrites are slower than the original query, the original query itself is returned (Line 32).

5.2 Input Prompt to MCTS

The root state in Algorithm 1 corresponds to the state just after the prompt is fed to the LLM. One way to use this algorithm is to execute it for all the various prompts discussed in the previous sections, and choose the rewrite that provides the best performance. This, however, is expensive both from the aspect of query rewrite time, as well as the number of LLM tokens used. To minimize these costs, given a user query, the LITHE workflow first selects the prompt yielding the most effective rewrite from among the techniques of Sections 3 and 4. It then employs this prompt to initiate the MCTS-based rewrite. In case no prompt provides a lower-cost rewrite, baseline Prompt 1 of Section 3 is used as the fallback option.

5.3 Performance

LITHE’s performance with MCTS-based rewrites fully matches the human target, yielding 10 CPRs and the maximum CSGM of 11.84. While these gains over just prompting may seem marginal, we wish make two observations: (1) MCTS extracts maximum improvement wherever possible, and (2) As shown in Section 7.5.2, its impact becomes significant for smaller LLMs like LLaMA.

6 Implementation Choices

In this section, we briefly discuss the design choices made in our implementation of LITHE.

6.1 LITHE Parameter Settings

The “*temperature*” parameter of GPT-4o, which ranges over [0,1], controls randomness of the model’s response. While higher values suit creative tasks requiring diverse and exploratory outputs, our objective is to obtain focused, deterministic answers. Thus, we set the temperature to 0, forcing the model to greedily sample the next token.

The hyperparameters used by LITHE for MCTS are as follows: The maximum number of iterations $iter_{max}$ is set to 8, expansion threshold θ is 0.7, and number of expansions k is 2. The values of c_{base} and c were set to 10 and 4, respectively. These settings were determined after an empirical evaluation of the various trade-offs, providing a robust balance between efficiency and quality.

Finally, we try a maximum of 5 times to fix, via prompt corrections, any rewrite that exhibits syntax errors (Section 2) – if this threshold is crossed, we drop the rewrite.

6.2 Query Equivalence Testing

We use a multi-stage approach, described below, to help the DBA test semantic equivalence between the original query and a recommended rewrite. As mentioned earlier, having the DBA in the loop is a common practice in commercial query advisory systems [17].

1. Result Equivalence via Sampling. We use a sampling-based approach to quickly test equivalence in the rewrite generation stages of the pipeline. The idea here is to execute the queries on several small samples of the database and verify equivalence based on the sample results. However, while this test is a necessary condition for query equivalence, it is not sufficient. That is, false positives may be present because the sampled database may not cover all the predicates featured in the query. To minimize this likelihood, we use a combination of (1) *correlated sampling* [50] for maintaining join integrity in the sample, (2) adjusting constants in the filter predicates to produce populated results, and (3) injecting query-specific synthetic tuples in the sample, similar to the XData mutant-killing tool [39], to cover predicate boundary conditions – the complete details are in the technical report [18].

2. Logic-based Equivalence. Although verifying the equivalence of a general pair of SQL queries is NP-complete [10], a variety of logic-based tools (e.g. Cosette[15], SQL-Solver [21], VeriEQL [26], QED [45]) are available for proving equivalence over restricted classes of queries. Once the least-cost rewrite is identified, LITHE uses QED [45] and SQLSolver [21] since these two approaches covered a larger set of queries compared to the alternatives. The advantage of such a logic-based approach is that it is definitive in outcome. Note that this rewrite has already passed the sampling-based tests described above.

3. Result Equivalence on the Entire Database. If the logic-based test is inconclusive, result equivalence is evaluated on the entire database itself. The DBA may choose to prematurely terminate this test in case the checking time becomes excessive.

6.3 Regression Identification

We use heuristics to predict whether the actual execution time of a given promising rewrite may turn out to be slower than the original query. The first heuristic is based on the SEER robust plan identification algorithm [23], which is predicated on the database engine providing a “force-plan” feature (e.g. [1]). This approach is based on the notion of *replacement safety*, wherein the replacement plan is always either better or within a small sub-optimality factor of the original optimizer-recommended choice. It was theoretically shown that a safe plan replacement at the *perimeter* of the selectivity space is sufficient to infer safe replacement in the interior of the space as well. The proof used a generic characterization of plan cost functions and the first and second derivatives of these functions at the perimeter. As a matter of efficiency, it was empirically found that instead of the entire perimeter, it was usually sufficient to consider safety just at the *corners*.

We use the above result to design a regression test as follows: let P^U and P^T be the plans corresponding to the original query, Q^U , and the recommended rewrite, Q^T , respectively. We construct *parametrized* versions of these queries, where the constants in the filter predicates are replaced by variables. Then, by

assigning appropriate values to these parameters, we construct queries that are located at the *corners* of the selectivity space. The plans P^U and P^T are forced at each of these corner locations, and if the rewrite's cost is lower than the original at *all* of them, the rewrite is deemed to be robust.

At this time, plan forcing is not available in all database engines. For such limited environments, we fall back to an alternative heuristic – we compare the query runtimes obtained on the series of executions carried out on sampled databases during semantic equivalence testing. If the rewrite's runtimes are lower in *all* of these intermediate executions, then it is estimated to be robust. We found this extrapolation heuristic to be effective in our experiments due to two reasons: First, the sampled databases, while relatively small compared to the original, are still large in absolute terms (1 GB or more). Second, since most rewrites are related to redundancy removals, a relative performance improvement obtained due to such a rule holds true irrespective of the data sizes.

7 Experimental Evaluation

In this section, we report on LITHE's performance profile. We first describe the experimental setup, including comparative baselines, query suites and evaluation platforms. Then we present the speedup results for both aggregate benchmark and individual queries, followed by characterization of the rewrite overheads in computational and financial terms. We finally discuss the impact of alternative platforms wrt database engine, database schema and LLMs.

Rewrite Baselines. We compare LITHE with a collection of contemporary rewrite techniques, collectively referred to as SOTA – the details of these techniques are provided in Section 9. Specifically, the SOTA collection consists of the following approaches:

- (1) Baseline LLM prompt [33]: This is Prompt 1 from Section 3.
- (2) Learned Rewrite [53], a purely rule-based rewriter.
- (3) LLM- R^2 [32], an LLM-guided rule-based rewriter.
- (4) GenRewrite [33], a purely LLM-based rewriter.

Given an input query, each of the SOTA approaches is independently invoked to perform a rewrite, and the rewrite with the *best* performance is used as the baseline for comparison. Note that these approaches may occasionally generate rewrites that are expected by the optimizer's costing module to regress the performance. For safety, we immediately discard such rewrites, similar to LITHE.

Query Set. Our evaluation in this paper primarily focuses on complex analytical queries from the standard TPC-DS decision-support benchmark [16], which models a retail supplier environment. The benchmark is used at its default size of 100 GB. As mentioned earlier, we focus on “slow queries” that take over 10 seconds to execute in their original form, operating in a cold-cache environment.

LITHE has also been evaluated on other benchmarks, including DSB [19], ARCHER [52], JOB [29] and StackOverflow [35]. The overall performance characteristics were found to be similar to those presented here – due to space limitations, we defer their details to the technical report [18].

Testbed. The experiments were carried out on the following data processing platform: Sandbox server with Intel(R) Xeon(R) CPU E5-1660, 32 GB RAM, and 12 TB HDD, running Ubuntu 22.04 LTS. A majority of the experiments used PostgreSQL v16 database engine and GPT-4o LLM for both LITHE and SOTA. Commercial engines and other LLMs are considered in Sections 7.4 and 7.5.

Metrics. For each rewrite technique, we identified the number of queries for which a CPR (cost productive rewrite with > 1.5 speedup) was constructed. Subsequently, we computed the CSGM (Cost Speedup Geometric Mean) and TSGM (Time Speedup Geometric Mean) performance obtained by each technique over the set of *all* CPRs (i.e. arising from either LITHE or SOTA). Cost speedups are computed relative to the native optimizer, and runtime speedups are measured as the ratio of original query runtime to rewritten query runtime in a cold-cache environment.

From the investment perspective, we measured the average rewrite time per query, and additionally for the LLM-based techniques, the number of tokens used in the rewrite process.

7.1 Rewrite Quality (Cost and Time)

7.1.1 Estimated Cost. LITHE produces a rewrite with a positive cost speedup ($> 1x$) for 46 of the 88 TPC-DS queries deemed to be slow by our threshold. Of these 46, there were 26 CPRs resulting in a *highly productive* CSGM of 11.5. On the other hand, SOTA delivers only 13 CPRs (out of 42 positive rewrites) with a CSGM of 6.1. All but one of the SOTA CPRs also feature in the LITHE CPRs, making the total number of CPRs considered to be 27. Of these 27, we were able to formally verify 11 using the logic-based tools, whereas the remaining 16 passed our statistical tests. Furthermore, we also manually verified the correctness of these rewritten queries.

A drill-down into the cost speedup performance at the granularity of individual queries is shown in Figure 8, which compares LITHE (orange bars) and SOTA (blue bars) on each of the 27 CPR queries – note that the cost speedups on the x -axis are tabulated on a $\log_{10}$ scale, and the queries are sequenced in decreasing order of LITHE speedup. The vertical dotted line at 1 represents the normalized baseline cost of the original query with the native optimizer, while the vertical line at 1.5 is the CPR threshold.

We first observe, gratifyingly, that rewrites are indeed capable of promising dramatic cost speedups over the native engine – take, for instance, Q41, which improves by a whopping *five orders-of-magnitude* for both SOTA and LITHE. This improvement in query performance is due to replacing the “WHERE (SELECT COUNT(*) from ...) > 0 ” clause with “WHERE EXIST (SELECT 1 from ...)” – the latter efficiently checks result existence in an inner subquery since it removes the costly aggregation function.

Second, in as many as 15 queries, LITHE's cost speedup *substantively exceeds* SOTA, while in the remainder it largely matches SOTA. In fact, for several queries (e.g. Q45, Q25, Q4) LITHE produces a highly beneficial CPR but SOTA returns the original query. The only case where SOTA is appreciably better is Q57 where it projects a large speedup but LITHE settles for the original query. And in Q88 and Q95, SOTA performs marginally better.

At this stage, one might expect that adding more rules to LITHE could bring it on par with SOTA for queries like Q57. However, we deliberately include only broad-brush rules in LITHE to ensure generalizability and efficiency. Moreover, as the following timing section shows, due to this conservative approach LITHE actually *outperforms* SOTA on these queries (Q57, Q88, Q95) at runtime!

We also analyzed queries where LITHE was unable to provide CPRs to determine whether there were common patterns that led to unsuccessful outcomes. Our audit indicated that the primary reasons were: (a) Structural Simplicity – for instance, flat SPJ queries without nesting that are already conducive to current optimizers (e.g. Q3, Q15, Q52), or (b) Structural Tightness – for instance, queries without repeated computations where the scope

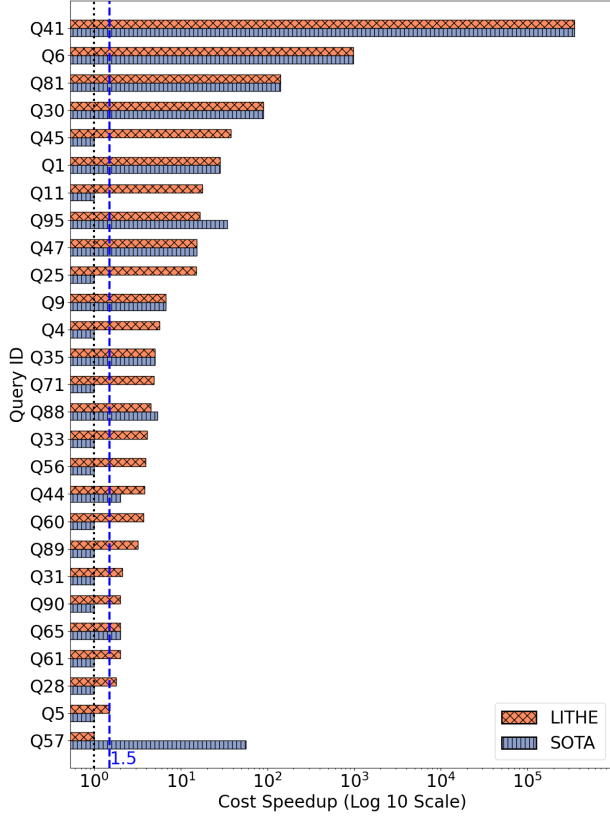


Figure 8: Plan Cost Speedups via Rewrites

for improvement is inherently limited (e.g. Q7, Q19, Q22). Further, some failures may be due to the *specific* LLM being used, and not the queries themselves (see Section 7.5.2). Therefore, one could consider implementing an *ensemble* of diverse LLMs to gain enhanced beneficial coverage.

7.1.2 Execution Time. Thus far, we had considered optimizer-estimated costs. We now move to wall-clock runtimes for query executions – Figure 9 shows the runtime speedups (on a $\log_{10}$ scale) obtained by LITHE and SOTA. We again observe several queries where substantial time benefits are achieved by the rewrites over the native engine, sometimes even exceeding *order-of-magnitude* benefits – for instance, LITHE improves Q45 by a huge factor of 700! Further, for 12 of the 26 queries, LITHE significantly outperforms SOTA – as a case in point, by **14** times in Q11. In the remainder, it matches SOTA, including as mentioned above, queries where SOTA’s optimizer costs were better.

From a modeling perspective, we see that the well-documented gap between optimizer predictions and actual run-times is prevalent in the rewrite space as well. On the one hand, there is Q45 where the projected speedup of 40 increases to 700 at runtime, whereas on the other, the 10^5 speedup for Q41 decreases to 200. But for SOTA, the reductions can be severe – a striking case in point is Q57, where SOTA actually causes *regression* despite a speedup projection of close to 100. However, the good news is that with LITHE, although the runtime speedups did not always match the projections, regressions were not encountered among the CPR rewrites thanks to the sampling-based checks. Overall, LITHE produces more robust rewrites resulting in a high TSGM of **13.2**, whereas SOTA only delivers TSGM of **4.9**.

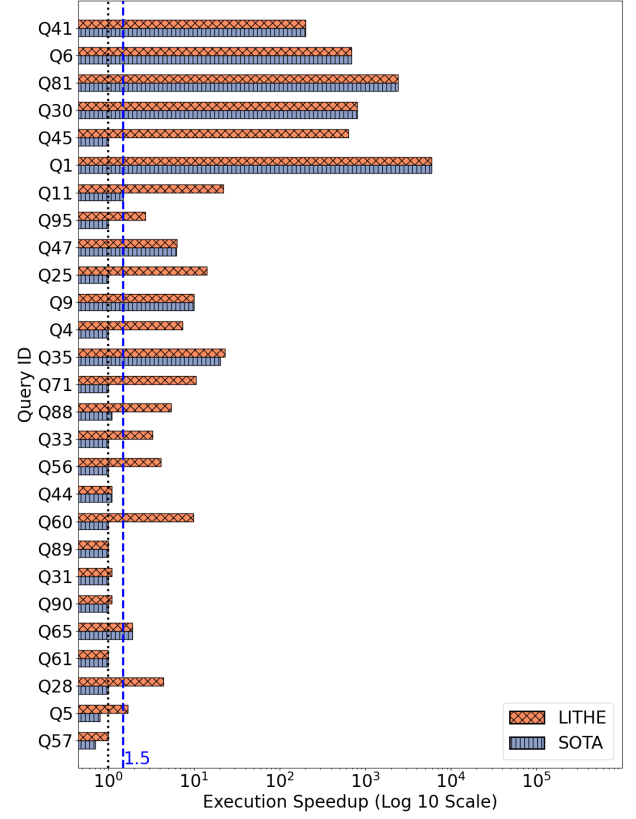


Figure 9: Execution Time Speedups via Rewrites

7.1.3 Reasoning. As a confirmatory exercise, we compared the LLM-generated explanation output by LITHE with our own manual analysis of the query plans generated for the original and rewritten queries. For example, consider Q45 – the provided explanation correctly lists the main reasons for the 700x speedup as “the use of CTEs”, “Reduction in data volume early on” due to pre-filtering based on join predicates, etc. Overall, we found that the explanations provided by LITHE matched our manual analysis of the plans, indicating that model-based reasoning is well aligned with human-backed reasoning in these scenarios.

7.2 Ablation Analysis of Rewrite Quality

7.2.1 Components of LITHE. A natural question at this stage is the role of the various techniques in LITHE towards achieving its large performance benefits. This analysis is captured in Table 5 which lists the CPRs for each technique when invoked in *isolation* (for MCTS, Prompt 1 is used as the seed) as well as the *cumulative* number of CPRs when the different techniques are combined in the order of listing.

Table 5: CPRs contributed by LITHE components

	# CPR	
	Isolated	Cumulative
Basic Prompts (Section 3)	15	15
Rules R1 – R4 (Section 4.1)	10	18
Rules R5, R6 (Section 4.2)	11	25
MCTS (Section 5)	11	26

We observe that the Basic Prompt ensemble and Redundancy Rules (R1 – R4) contribute to around two-thirds of the CPRs (18/26). When the statistics-infused Rules (R5, R6) are added, this number jumps to 25. Finally, for MCTS when using the best prompt as seed, an additional CPR is obtained cumulatively and the cost speedup of a pre-existing CPR is also improved.

7.2.2 Database-Sensitive Prompts. Since MCTS explores the search space at a fine granularity, one could ask whether just the Basic Prompts in conjunction with MCTS would suffice to provide good performance. The motivation is that it would relieve us from using the database-sensitive rules R1–R6 which incur significant computational and financial overheads. When this experiment was conducted, the CSGM dropped precipitously to a paltry 5, a far cry from the 11.5 obtained with the database-sensitive rules. These results highlight the need to reflect database awareness for effective query rewriting, and not rely solely on prior LLM knowledge.

7.3 Rewrite Overheads (Time/Money)

Having established the performance benefits of rewrites, we now turn our attention to their time and financial overheads.

7.3.1 Transformation Time. We first look at the end-to-end time required to perform a rewrite (i.e. to run the entire pipeline shown in Figure 3). Table 6 shows the average processing time per CPR query, where we see that the LITHE rewrite process takes a few minutes. However, note that this investment may be acceptable in deployment given that the execution benefits typically far outweigh the compilation overheads. For instance, with Q11, the original query took nearly *an hour* to complete, whereas the LITHE rewrite executed in under 3 *minutes*. Further, many applications tend to use a set of canned queries which are run thousands of times. Thus, even a large one-time investment can be easily recovered over repeat executions of such queries.

Table 6: Rewrite Overheads of LITHE and SOTA

	Avg. Time (min)	Avg. Tokens	Avg. Cost (USD)
LITHE	5	18427	0.045
SOTA	1.7	20076	0.050

Notwithstanding the above, we also observe that LITHE is considerably slower than SOTA in producing rewrites. A drill-down showed the lion’s share of the time is taken by the initial prompt ensemble and the final MCTS module. We explore a classifier-based approach to improve the efficiency of the former in the Section 7.3.3.

7.3.2 Monetary Outlay. The average number of LLM tokens required by LITHE and SOTA, and their associated financial costs¹, are also shown in Table 6. The good news is that the inference charges per query are just a few cents, making rewriting practical from a deployment perspective.

7.3.3 Rule Selection Classifier. Reducing the query rewrite time while still obtaining the same performance would be possible if we could directly use the MCTS-driven rewrite with the appropriate prompt. Towards this end, we build a classifier to pick the most appropriate rewrite rule for a given input query. Specifically, the classifier identifies which, if any, of Rules R1–R6 is appropriate for a given query. If none are appropriate, then it

¹At the time of writing, GPT-4o cost USD 2.5 per million tokens.

falls back to just the set of basic prompts to identify the best prompt to be given as input to the MCTS module. In addition to reducing the rewrite times, using a classifier can also reduce the financial costs of the rewrite.

We design an LLM based classifier to accomplish this as follows: The LLM is given the rewrite rules discussed so far, and additionally, for each rule, an example demonstrating when the rule can be applied and a counter-example demonstrating when the rule cannot be applied. For the database schema and statistics-based rules, the relevant information is also fed to the classifier so that it can make an informed decision. Then, given an input query, the classifier is tasked with selecting the most appropriate rewrite rule.

Table 7: Impact of Classifier (TPC-DS)

Metrics	Without Classifier	With Classifier
# CPR	26	23
CSGM	11.5	8.5
Avg. Tokens	18427	16003
Avg. Time	5 min	2.4 min

Table 7 compares the performance of LITHE with and without the classifier. The time overheads do visibly go down by about 52 percent, and the tokens by about 13 percent. However, there is a price to be paid – the CPR is reduced to 23 and the CSGM comes down to 8.5. In our future work, we plan to look into whether a better tradeoff is achievable between quality and overheads.

7.4 Commercial DBMS

A legitimate question could be whether the rewrites made amends for the PostgreSQL optimizer but may fail to be useful in highly-engineered database engines. To evaluate this issue, we performed TPC-DS rewrites on a pair of popular commercial DBMS, which we refer to as **OptA** and **OptB**.

Table 8: Performance on Commercial Database Engines

	# CPR		CSGM		TSGM	
	OptA	OptB	OptA	OptB	OptA	OptB
LITHE	12	9	3.6	4.1	2.1	1.9
SOTA	3	5	1.5	1.3	1.4	1.2

The performance of LITHE and SOTA on these two systems is shown in Table 8, with LITHE continuing to do better than SOTA. Despite the apparent lack of optimization headroom, LITHE still produces 12 and 9 CPRs resulting in a healthy CSGM of **3.6** and **4.1**, respectively. Further, the TSGM provided by these rewrites are a useful **2.1** and **1.9**, respectively.

Interestingly, although we did not observe any regressions with PostgreSQL, a few did surface in the commercial systems. Nevertheless, our regression identification mechanisms effectively caught these brittle rewrites. As a case in point, a promising rewrite estimated by one of the commercial optimizers for Q23, actually takes 37 minutes to complete as compared to 18 minutes for the original query – this doubling slowdown was successfully flagged by the SEER heuristic (Section 6.3), and the rewrite was abandoned.

The above results suggest LITHE has a useful role to play in industrial environments. From a different perspective, a company building a new database engine could use LITHE to non-invasively overcome the limitations of early versions of its optimizer.

7.5 Dependence on LLM

7.5.1 Impact of Training. An interesting question to ask now is whether the performance benefits seen thus far could be an artifact of GPT-4o having already been trained well on the TPC-DS benchmark, which is prominent in the public domain.

To investigate this issue, we ran LITHE on two datasets that were confirmed to be *unknown* to the GPT-4o version used in our evaluation. The first is the recently released Football benchmark [25] for Text-to-SQL evaluation. From the query suite, which largely consists of simple SPJ and Union queries, we selectively picked the few complex queries. Further, we created our own complex queries to include in the test suite. On this enriched workload, LITHE on PostgreSQL produced 11 CPRs with CSGM of 12 and TSGM of 1.5, while SOTA gave 6 CPRs with CSGM of 3 and TSGM of 1.1.

The second dataset is a proprietary customer benchmark used extensively by an industrial development team. For this environment, LITHE on a commercial engine produced 8 CPRs with a CSGM of 2 and a TSGM of 3.3. Note that these improvements are achieved *despite* this engine having been fine-tuned on this workload over an extended period. Moreover, the numbers are consistent with those reported for TPC-DS in Section 7.4.

7.5.2 Impact of Model. In our concluding experiment, we evaluated LITHE’s performance on the LLaMA 3.1 70 billion parameter instruct model, substantially smaller compared to GPT-4o, which is said to have several hundred billion parameters [8]. To attain practical inference times, the model was loaded using a low 4-bit quantization. Further, to ensure reproducibility and deterministic answers, the *do_sample* parameter was set to *False*, forcing the LLM to perform greedy decoding. To make up for the huge reduction in model parameters as compared to GPT-4o, we include up to two example demonstrations for each rule-based prompt.

For this environment, Table 9 shows LITHE’s performance on the TPC-DS queries with and without MCTS. Although certainly lower than the corresponding numbers with GPT-4o (Section 7.1), it is encouraging to see that, in absolute terms, significant performance benefits can be obtained for most queries, especially with MCTS support. So, the message is that smaller models can also be fruitfully used in real-world environments.

Table 9: LITHE Rewrite Performance with LLaMA

	# CPR	CSGM	TSGM
LLaMA without MCTS	18	5.6	6.5
LLaMA with MCTS	22	8.5	10.9

Finally, we also evaluated LITHE on **Gemini 2.5 Flash**. Here, LITHE produced 17 CPR with CSGM of 7.2 and TSGM of **5.8**. Although not as strong as GPT-4o, these results highlight the potency of LITHE’s recommendations across a range of LLM platforms.

8 The Road Ahead

We now present a few observations with implications for the future design and deployment of rewriting tools.

8.1 Rewrite Space Coverage by LLMs

Given the decades-long research on database query optimization, we expected the potential for performance improvement via rewriting to be limited. What came as a surprise was the substantial scope for improvement still available, as showcased by the

```
SELECT CAST(
  SUM( CASE WHEN time_dim.t_hour BETWEEN 8 AND 9
            THEN 1 ELSE 0 END) AS DECIMAL(15, 4)
) / CAST(
  SUM( CASE WHEN time_dim.t_hour BETWEEN 19 AND 20
            THEN 1 ELSE 0 END) AS DECIMAL(15, 4)
) AS am_pm_ratio
FROM web_sales
JOIN household_demographics
  ON ws_ship_hdemo_sk = household_demographics.hd_demo_sk
JOIN time_dim ON ws_sold_time_sk = time_dim.t_time_sk
JOIN web_page ON ws_web_page_sk = web_page.wp_web_page_sk
WHERE household_demographics.hd_dep_count = 6
  AND web_page.wp_char_count BETWEEN 5000 AND 5200
  AND (time_dim.t_hour BETWEEN 8 AND 9
       OR time_dim.t_hour BETWEEN 19 AND 20)
ORDER BY am_pm_ratio
limit 100;
```

Figure 10: Rewritten TPC-DS Q90

large CSGM and TSGM values, even on commercial platforms. These results suggest that LLMs explore optimization spaces that are well outside the purview of contemporary database engines. Specifically, rewrites in the query space appear to guide the optimizer to explore fresh regions of the plan space not evaluated with the original query formulation. Further, this enhanced space could be augmented, in a two-stage process, with the recent proposals for LLM-based “plan hints” that steer the optimizer in fruitful directions within a plan space [11]. We plan to explore these space relationships and options in our future work.

8.2 Rewrite Migration to Optimizer

The above demonstrated the potent exploratory power of LLMs. But from an overheads perspective, such rewrites should ideally be within the optimizer’s native search space rather than recommended from outside. Therefore, it would be a useful exercise to try and distill fresh optimization rules from these instances, leveraging the extensibility features of contemporary optimizers [20] to facilitate their incorporation in existing systems.

On the flip side, there appears to be an “impedance mismatch” against such integration for certain classes of rewrites. For example, consider the TPC-DS Q90 rewrite in Figure 10. The original query individually computed AM (morning) sales and PM (evening) sales, which were then used to compute the AM to PM ratio. The rewrite, however, extracted all relevant rows in one shot and computed the ratio using CASE statements – encoding such transformations as generic rules in the optimizer appears challenging, given the combinatorial ways in which such transformations can occur.

Therefore, a fruitful area of future research could be achieving a middle-ground between the disparate world-views of LLMs and traditional optimizers.

8.3 Agentic LLMs for Query Rewriting

An effective way to extend LITHE is with an agentic LLM that actively interacts with the database environment. This would allow a “LITHE agent” to leverage database-related “tools” such as the query optimizer’s cardinality and cost estimation modules, as well as rewrite-related tools such as rule generation, semantic validation, and regression checking. A memory store could be attached to the agent to log prior interactions with users and rewrite rules learnt over time, thus supporting continuous learning. The agent decides the order in which these tools are invoked and makes appropriate choices during the rewrite process.

8.4 Scope of Semantic Equivalence Tools

As seen in the experiments section, logic-based query equivalence testing covers industrial-strength queries only to a limited extent. On the other hand, while it is highly likely that the statistics-verified rewrites are valid, it still requires the DBA to make a final call on the correctness. This limitation restricts the use of LITHE in a fully automated scenario, i.e., as a direct preprocessor to the query engine. Therefore, a key challenge is to improve logic-based coverage.

9 Related Work

Rule-based SQL rewriting. Most of the recent work on SQL query rewriting is rule-based [12, 14, 37, 47, 48, 53]. For instance, WeTune [47] uses a rule generator to enumerate a set (up to a maximum size) of logically valid plans for a given query to create new rewrite rules, and uses an SMT solver to prove the correctness of the generated rules. While this approach can generate a large set of new rewrite rules, it often fails in coming up with transformation rules for complex queries due to the computational overheads of verifying rule correctness. As such, it is unable to rewrite any of the TPC-DS queries [22, 33].

Learned Rewrite [53] uses the existing Calcite [13] rules and aims to learn the optimal subset of rules along with the order in which they must be applied. Since the rewrite search space grows exponentially with the number of rules, it uses the MCTS scheme to efficiently navigate this space and find the rewritten query with maximum cost reduction. This is in contrast to LITHE where MCTS is used to search over the output token space of an LLM in order to prevent LLM hallucinations. Specifically, our use of MCTS ensures that the LLM correctly follows the prompt instructions.

LLM-R² [32] is also rule-based but takes a different approach to identify the order for rewrite rule applications: it uses an LLM to find the best Calcite rules and the order in which to apply them to improve the query performance. R-Bot [42] also leverages an LLM to optimize the order of Calcite rules, but employs advanced contemporary techniques such as retrieval-augmented generation (RAG) and step-by-step self-reflection to improve the outcomes.

Query Booster [12] implements human-centered rewriting – it provides an interface to specify rules using an expressive rule language, which it generalizes to create rewrite rules to be applied on the query. There are also rule-based rewrite approaches designed for specific types of rewrites such as optimizing correlated window aggregations [48] and common expression elimination [14].

All of the above approaches operate via the query *plan space*, which can restrict the kind of rewrites that can be accomplished. Whereas, LITHE uses a small set of general rewrite rules that work directly in the *query space*.

LLM-based rewriting. GenRewrite [33] is the first LLM-based approach to use the LLM for end-to-end query rewriting. Instead of using predefined rules from Calcite [13], they employ the LLM to create Natural Language Rewrite Rules (NLRs) to be used as hints, and perform several iterations of prompting to get the rewritten query. They show that LLMs can outperform rule-based approaches due to their ability to understand contexts, and demonstrate a significant improvement in query rewriting compared to prior methods.

A limitation, however, is that LLM-generated rewrite rules often fail to generalize beyond specific query pairs. Even when

generalized rules are present, LLMs struggle to apply rules correctly if not provided with accompanying examples. Finally, since LLMs are unaware of the underlying database, it fundamentally restricts their ability to produce efficient metadata-aware rules.

LLMs for Database Modules. LLM technologies have been advocated for a variety of database modules. For instance, they have been extensively used for Text-to-SQL (T2S) transformations [4, 5, 31, 43, 49, 54]. Very recently, statistical metadata was leveraged to also improve the T2S generation process [40]. The main focus of these techniques is to correctly ascertain the information necessary to formulate the SQL query. On the other hand, our goal is on improving the performance of an existing SQL query. Thus, unlike T2S where the input text is inherently ambiguous, SQL queries are precisely defined, and therefore equivalence to a precise ground-truth has to be provably maintained.

In recent times, LLMs have also been considered for a variety of database modules, including plan-hinting [11], join-order optimization [44], index selection [54], data pipelines [28], data management [30], and multi-modal data optimization [46]. These approaches can be used in conjunction with LITHE since they address orthogonal segments of the query processing pipeline.

10 Conclusions and Future Work

We investigated how the latent power of LLM technologies can be productively materialized in the context of SQL-to-SQL rewriting. Our study progressively infused database domain knowledge, such as redundancy removal rules and schematic+statistical metadata, into the LLM prompts. Further, the output telemetry of LLMs, in the form of token probabilities, was used to signal situations where the LLM lacked confidence, triggering exploration of a larger search space. Finally, a combination of logic-based and statistical tests was employed to verify the equivalence of the rewrites.

An empirical evaluation over common database benchmarks showed that rewriting is a potent mechanism to improve query performance. In fact, even order-of-magnitude speedups were routinely achieved with regard to both abstract costing and execution times. However, our results also showed a significant semantic distance between foundation models and query optimizers, with regard to both scope and precision, which would have to be bridged to fully leverage the latent power of LLMs. Further, our focus here was primarily on prompting-based strategies – in our future research, we plan to investigate how domain-specific *fine-tuning* could be leveraged to provide GPT-4o-like rewrites on small open models.

Acknowledgments

We thank the members of Azure SQL India and MSR Redmond for valuable technical inputs. We are grateful to students of Database Systems Lab at IISc Bangalore, especially Manish Kesarwani, for their constructive reviews of the manuscript. This research work was initiated at MSR India during a sabbatical of the third author, and carried forward during student internships at MSR India of the first two authors.

Artifacts

A supplementary document to the work presented here can be found at [18]. It includes: (a) results on additional database benchmarks; (b) full details on correlated sampling techniques; and (c) rule demonstration examples provided to the LLM.

References

- [1] 2012. SqlServer XML Plan forcing. <http://msdn2.microsoft.com/en-us/library/ms189298.aspx>
- [2] 2019. View SQL Queries From Your Code With Prefix. <https://stackify.com/view-sql-with-prefix/>
- [3] 2023. PostgreSQL Release 16. www.postgresql.org/docs/16/release-16.html
- [4] 2024. Defog SQL-Eval. <https://github.com/defog-ai/sql-eval>
- [5] 2024. Defog SQLCoder. github.com/defog-ai/sqlcoder
- [6] 2024. GroupBy with joined table produces sub optimal query. github.com/npgsql/efcore.pg/issues/3160
- [7] 2024. MySQL 8.4 Reference Query Manual – The Slow QUery Log. <https://dev.mysql.com/doc/refman/8.4/en/slow-query-log.html>
- [8] 2024. OpenAI's GPT 4o vs GPT 4o mini: Which AI model to use and why. <https://timesofindia.indiatimes.com/technology/tech-tips/openai-gpt-4o-vs-gpt-4o-mini-which-ai-model-to-use-and-why/articleshow/111927368.cms>
- [9] Rafi Ahmed, Allison Lee, Andrew Witkowski, Dinesh Das, Hong Su, Mohamed Zait, and Thierry Cruanes. 2006. Cost-based query transformation in Oracle. In *Proceedings of the 32nd International Conference on Very Large Data Bases (Seoul, Korea) (VLDB '06)*. VLDB Endowment, 1026–1036.
- [10] Alfred V. Aho, Yehoshua Sagiv, and Jeffrey D. Ullman. 1979. Equivalences Among Relational Expressions. *SIAM J. Comput.* 8, 2 (1979), 218–246. doi:10.1137/0208017
- [11] Peter Akiyamen, Zixuan Yi, and Ryan Marcus. 2024. The Unreasonable Effectiveness of LLMs for Query Optimization. *CoRR* abs/2411.02862 (2024). doi:10.48550/ARXIV.2411.02862 arXiv:2411.02862
- [12] Qiushi Bai, Sadeem Alsudais, and Chen Li. 2023. QueryBooster: Improving SQL Performance Using Middleware Services for Human-Centered Query Rewriting. *Proc. VLDB Endow.* 16, 11 (2023), 2911–2924. doi:10.14778/3611479.3611497
- [13] Edmon Begoli, Jesús Camacho-Rodríguez, Julian Hyde, Michael J. Mior, and Daniel Lemire. 2018. Apache Calcite: A Foundational Framework for Optimized Query Processing Over Heterogeneous Data Sources. In *Proceedings of the 2018 International Conference on Management of Data, SIGMOD Conference 2018, Houston, TX, USA, June 10-15, 2018*, Gautam Das, Christopher M. Jermaine, and Philip A. Bernstein (Eds.). ACM, 221–230. doi:10.1145/3183713.3190662
- [14] Nicolas Bruno, Johnny Debrodt, Chujun Song, and Wei Zheng. 2022. Computation Reuse via Fusion in Amazon Athena. In *38th IEEE International Conference on Data Engineering, ICDE 2022, Kuala Lumpur, Malaysia, May 9-12, 2022*. IEEE, 1610–1620. doi:10.1109/ICDE53745.2022.00166
- [15] Shumo Chu, Chenglong Wang, Konstantin Weitz, and Alvin Cheung. 2017. Cosette: An Automated Prover for SQL. In *8th Biennial Conference on Innovative Data Systems Research, CIDR 2017, Chaminade, CA, USA, January 8-11, 2017, Online Proceedings*. www.cidrdb.org. <http://cidrdb.org/cidr2017/papers/p51-chu-cidr17.pdf>
- [16] The Transaction Processing Performance Council. 2006. TPC Benchmark™ DS (TPC-DS). In *TPC Benchmark DS (Decision Support)*. www.tpc.org/tpcds/
- [17] Benoît Dageville, Dinesh Das, Karl Dias, Khaled Yagoub, Mohamed Zait, and Mohamed Ziauddin. 2004. Automatic SQL Tuning in Oracle 10g. In *(e)Proceedings of the Thirtieth International Conference on Very Large Data Bases, VLDB 2004, Toronto, Canada, August 31 - September 3 2004*, Mario A. Nascimento, M. Tamer Özsu, Donald Kossmann, Renée J. Miller, José A. Blakeley, and K. Bernhard Schiefer (Eds.). Morgan Kaufmann, 1098–1109. doi:10.1016/B978-012088469-8.50096-6
- [18] Sriram Dharwada, Himanshu Devrani, Jayant R. Haritsa, and Harish Doraiswamy. 2025. Query Rewriting via LLMs. *CoRR* abs/2502.12918 (2025). doi:10.48550/ARXIV.2502.12918 arXiv:2502.12918
- [19] Bailu Ding, Surajit Chaudhuri, Johannes Gehrke, and Vivek R. Narasayya. 2021. DSB: A Decision Support Benchmark for Workload-Driven and Traditional Database Systems. *Proc. VLDB Endow.* 14, 13 (2021), 3376–3388. doi:10.14778/3484224.3484234
- [20] Bailu Ding, Vivek Narasayya, and Surajit Chaudhuri. 2024. Extensible Query Optimizers in Practice. *Foundations and Trends® in Databases* 14, 3-4 (2024), 186–402. doi:10.1561/19000000077
- [21] Haoran Ding, Zhaoguo Wang, Yicun Yang, Dexin Zhang, Zhenglin Xu, Haibo Chen, Ruzica Piskac, and Jinyang Li. 2023. Proving Query Equivalence Using Linear Integer Arithmetic. *Proc. ACM Manag. Data* 1, 4 (2023), 227:1–227:26. doi:10.1145/3626768
- [22] Rui Dong, Jie Liu, Yuxuan Zhu, Cong Yan, Barzan Mozafari, and Xinyu Wang. 2023. SlabCity: Whole-Query Optimization using Program Synthesis. *Proc. VLDB Endow.* 16, 11 (2023), 3151–3164. doi:10.14778/3611479.3611515
- [23] Harish Doraiswamy, Pooja N. Darera, and Jayant R. Haritsa. 2008. Identifying robust plans through plan diagram reduction. *Proc. VLDB Endow.* 1, 1 (2008), 1124–1140. doi:10.14778/1453856.1453976
- [24] Ekaterina Fadeeva, Aleksandr Rubashevskii, Artem Shelmanov, Sergey Petrakov, Haonan Li, Hamdy Mubarak, Evgenii Tsybalov, Gleb Kuzmin, Alexander Panchenko, Timothy Baldwin, Preslav Nakov, and Maxim Panov. 2024. Fact-Checking the Output of Large Language Models via Token-Level Uncertainty Quantification. In *Findings of the Association for Computational Linguistics, ACL 2024, Bangkok, Thailand and virtual meeting, August 11-16, 2024*, Lun-Wei Ku, Andre Martins, and Vivek Srikumar (Eds.). Association for Computational Linguistics, 9367–9385. doi:10.18653/V1/2024.FINDINGS-ACL.558
- [25] Jonathan Fürst, Catherine Kosten, Farhad Nooralhazadeh, Yi Zhang, and Kurt Stockinger. 2025. Evaluating the Data Model Robustness of Text-to-SQL Systems Based on Real User Queries. In *Proceedings 28th International Conference on Extending Database Technology, EDBT 2025, Barcelona, Spain, March 25-28, 2025*, Alkis Simitis, Bettina Kemme, Anna Queral, Oscar Romero, and Petar Jovanovic (Eds.). OpenProceedings.org, 158–170. doi:10.48786/EDBT.2025.13
- [26] Yang He, Pinhan Zhao, Xinyu Wang, and Yuepeng Wang. 2024. VeriEQL: Bounded Equivalence Verification for Complex SQL Queries with Integrity Constraints. *CoRR* abs/2403.03193 (2024). doi:10.48550/ARXIV.2403.03193 arXiv:2403.03193
- [27] Hibernate 2025. Hibernate ORM. <https://hibernate.org/orm/>
- [28] Sylvio Barbon Junior, Paolo Ceravolo, Sven Groppe, Mustafa Jarrar, Samira Maghool, Florence Sèdes, Soror Sahri, and Maurice van Keulen. 2024. Are Large Language Models the New Interface for Data Pipelines?. In *Proceedings of the International Workshop on Big Data in Emergent Distributed Environments, BiDEDE 2024, Santiago, Chile, June 9-15, 2024*, Philippe Cudré-Mauroux, Andrea Kö, and Robert Wrembel (Eds.). ACM, 6:1–6:6. doi:10.1145/3663741.3664785
- [29] Viktor Leis, Andrey Gubichev, Atanas Mirchev, Peter A. Boncz, Alfons Kemper, and Thomas Neumann. 2015. How Good Are Query Optimizers, Really? *Proc. VLDB Endow.* 9, 3 (2015), 204–215. doi:10.14778/2850583.2850594
- [30] Guoliang Li, Xuanhe Zhou, and Xinyang Zhao. 2024. LLM for Data Management. *Proc. VLDB Endow.* 17, 12 (2024), 4213–4216. doi:10.14778/3685800.3685838
- [31] Jinyang Li, Binyuan Hui, Ge Qu, Jiaxi Yang, Binhua Li, Bowen Li, Bailin Wang, Bowen Qin, Ruiying Geng, Nan Huo, Xuanhe Zhou, Chenhao Ma, Guoliang Li, Kevin Chen-Chuan Chang, Fei Huang, Reynold Cheng, and Yongbin Li. 2023. Can LLM Already Serve as A Database Interface? A Blg Bench for Large-Scale Database Grounded Text-to-SQLs. In *Advances in Neural Information Processing Systems 36: Annual Conference on Neural Information Processing Systems 2023, NeurIPS 2023, New Orleans, LA, USA, December 10 - 16, 2023*, Alice Oh, Tristan Naumann, Amir Globerson, Kate Saenko, Moritz Hardt, and Sergey Levine (Eds.). http://papers.nips.cc/paper_files/paper/2023/hash/83fc8fab1710363050bbd1d4b8cc0021-Abstract-Datasets_and_Benchmarks.html
- [32] Zhaodonghui Li, Haitao Yuan, Huiming Wang, Gao Cong, and Lidong Bing. 2024. LLM-R2: A Large Language Model Enhanced Rule-based Rewrite System for Boosting Query Efficiency. *Proc. VLDB Endow.* 18, 1 (2024), 53–65. <https://www.vldb.org/pvldb/vol18/p53-yuan.pdf>
- [33] Jie Liu and Barzan Mozafari. 2024. Query Rewriting via Large Language Models. *CoRR* abs/2403.09060 (2024). doi:10.48550/ARXIV.2403.09060 arXiv:2403.09060
- [34] G. Lohman. 2014. Is Query Optimization a Solved Problem? wp.sigmod.org/?p=1075
- [35] Ryan Marcus. 2021. Stack dataset. rmarcus.info/stack.html
- [36] Microsoft 2025. Entity Framework. <https://learn.microsoft.com/en-us/ef/>
- [37] Hamid Pirahesh, Joseph M. Hellerstein, and Waqar Hasan. 1992. Extensible/Rule Based Query Rewrite Optimization in Starburst. In *Proceedings of the 1992 ACM SIGMOD International Conference on Management of Data, San Diego, California, USA, June 2-5, 1992*, Michael Stonebraker (Ed.). ACM Press, 39–48. doi:10.1145/130283.130294
- [38] Martin L. Puterman. 1994. *Markov Decision Processes: Discrete Stochastic Dynamic Programming*. Wiley. doi:10.1002/9780470316887
- [39] Shetal Shah, S. Sudarshan, Suhas Kajbaje, Sandeep Patidar, Bhanu Pratap Gupta, and Devang Vira. 2011. Generating test data for killing SQL mutants: A constraint-based approach. In *Proceedings of the 27th International Conference on Data Engineering, ICDE 2011, April 11-16, 2011, Hannover, Germany*, Serge Abiteboul, Klemens Böhm, Christoph Koch, and Kian-Lee Tan (Eds.). IEEE Computer Society, 1175–1186. doi:10.1109/ICDE.2011.5767876
- [40] Vladislav Shkapenyuk, Divesh Srivastava, Theodore Johnson, and Parisa Ghane. 2025. Automatic Metadata Extraction for Text-to-SQL. arXiv:2505.19988 [cs.DB] <https://arxiv.org/abs/2505.19988>
- [41] David Silver, Thomas Hubert, Julian Schrittwieser, Ioannis Antonoglou, Matthew Lai, Arthur Guez, Marc Lanctot, Laurent Sifre, Dharshan Kumaran, Thore Graepel, Timothy P. Lillicrap, Karen Simonyan, and Demis Hassabis. 2017. Mastering Chess and Shogi by Self-Play with a General Reinforcement Learning Algorithm. *CoRR* abs/1712.01815 (2017). arXiv:1712.01815 <http://arxiv.org/abs/1712.01815>
- [42] Zhaoyan Sun, Xuanhe Zhou, and Guoliang Li. 2024. R-Bot: An LLM-based Query Rewrite System. *CoRR* abs/2412.01661 (2024). doi:10.48550/ARXIV.2412.01661 arXiv:2412.01661
- [43] Shayan Talaie, Mohammadreza Pourreza, Yu-Chen Chang, Azalia Mirhoseini, and Amin Saberi. 2024. CHESS: Contextual Harnessing for Efficient SQL Synthesis. *CoRR* abs/2405.16755 (2024). doi:10.48550/ARXIV.2405.16755 arXiv:2405.16755
- [44] Jie Tan, Kangfei Zhao, Rui Li, Jeffrey Xu Yu, Chengzhi Piao, Hong Cheng, Helen Meng, Deli Zhao, and Yu Rong. 2025. Can Large Language Models Be Query Optimizer for Relational Databases? *CoRR* abs/2502.05562 (2025). doi:10.48550/ARXIV.2502.05562 arXiv:2502.05562
- [45] Shuxian Wang, Sicheng Pan, and Alvin Cheung. 2024. QED: A Powerful Query Equivalence Decoder for SQL. *Proc. VLDB Endow.* 17, 11 (2024), 3602–3614. doi:10.14778/3681954.3682024
- [46] Yifan Wang, Haodi Ma, and Daisy Zhe Wang. 2024. No more optimization rules: LLM-enabled policy-based multi-modal query optimizer. *CoRR* abs/2403.13597 (2024). doi:10.48550/ARXIV.2403.13597 arXiv:2403.13597
- [47] Zhaoguo Wang, Zhou Zhou, Yicun Yang, Haoran Ding, Gansen Hu, Ding Ding, Chuzhe Tang, Haibo Chen, and Jinyang Li. 2022. WeTune: Automatic Discovery and Verification of Query Rewrite Rules. In *Proceedings of the 2018*

- International Conference on Management of Data, SIGMOD Conference 2022, Philadelphia, PA, USA, June 12 - 17, 2022*, Zachary G. Ives, Angela Bonifati, and Amr El Abbadi (Eds.). ACM, 94–107. doi:10.1145/3514221.3526125
- [48] Wentao Wu, Philip A. Bernstein, Alex Raizman, and Christina Pavlopoulou. 2022. Factor Windows: Cost-based Query Rewriting for Optimizing Correlated Window Aggregates. In *38th IEEE International Conference on Data Engineering, ICDE 2022, Kuala Lumpur, Malaysia, May 9-12, 2022*. IEEE, 2722–2734. doi:10.1109/ICDE53745.2022.00249
- [49] Siqiao Xue, Caigao Jiang, Wenhui Shi, Fangyin Cheng, Keting Chen, Hongjun Yang, Zhiping Zhang, Jianshan He, Hongyang Zhang, Ganglin Wei, Wang Zhao, Fan Zhou, Danrui Qi, Hong Yi, Shaodong Liu, and Faqiang Chen. 2023. DB-GPT: Empowering Database Interactions with Private Large Language Models. *CoRR* abs/2312.17449 (2023). doi:10.48550/ARXIV.2312.17449 arXiv:2312.17449
- [50] Feng Yu, Wen-Chi Hou, Cheng Luo, Dunren Che, and Mengxia Zhu. 2013. CS2: a new database synopsis for query estimation. In *Proceedings of the ACM SIGMOD International Conference on Management of Data, SIGMOD 2013, New York, NY, USA, June 22-27, 2013*, Kenneth A. Ross, Divesh Srivastava, and Dimitris Papadias (Eds.). ACM, 469–480. doi:10.1145/2463676.2463701
- [51] Shun Zhang, Zhenfang Chen, Yikang Shen, Mingyu Ding, Joshua B. Tenenbaum, and Chuang Gan. 2023. Planning with Large Language Models for Code Generation. In *The Eleventh International Conference on Learning Representations, ICLR 2023, Kigali, Rwanda, May 1-5, 2023*. OpenReview.net. <https://openreview.net/forum?id=Lr8cOotYbfl>.
- [52] Danna Zheng, Mirella Lapata, and Jeff Z. Pan. 2024. Archer: A Human-Labeled Text-to-SQL Dataset with Arithmetic, Commonsense and Hypothetical Reasoning. In *Proceedings of the 18th Conference of the European Chapter of the Association for Computational Linguistics, EACL 2024 - Volume 1: Long Papers, St. Julian's, Malta, March 17-22, 2024*, Yvette Graham and Matthew Purver (Eds.). Association for Computational Linguistics, 94–111. <https://aclanthology.org/2024.eacl-long.6>
- [53] Xuanhe Zhou, Guoliang Li, Jianming Wu, Jiesi Liu, Zhaoyan Sun, and Xinning Zhang. 2023. A Learned Query Rewrite System. *Proc. VLDB Endow.* 16, 12 (2023), 4110–4113. doi:10.14778/3611540.3611633
- [54] Xuanhe Zhou, Zhaoyan Sun, and Guoliang Li. 2024. DB-GPT: Large Language Model Meets Database. *Data Sci. Eng.* 9, 1 (2024), 102–111. doi:10.1007/S41019-023-00235-6